PYTHON FOR BEGINNERS · SESSION 18

Pandas Introduction

Series, DataFrame and creating data.

Recap

Ask the class:

- What does `axis=0` mean in `a.sum(axis=0)`?
- What does `np.where(marks >= 50, "Pass", "Fail")` do?
- What is broadcasting?

What is pandas?

pandas is the main Python package for **tables of data**, like an Excel sheet inside Python. It sits on top of NumPy and adds **labels**: names for rows and columns.

- Read data from **CSV**, Excel and more.
- Clean it, filter it, sort it and summarise it.
- Used everywhere in **data analysis** and machine learning.

Install it once on your computer (terminal)

```
pip install pandas
```

By convention we import it as `pd`. pandas has two main building blocks:

NAME	WHAT IT IS	THINK OF IT AS
Series	one column of data with labels	one column of a spreadsheet
DataFrame	a whole table: many Series side by side	the whole spreadsheet

```
import pandas as pd

print(pd.__name__)
print(type(pd.Series([1, 2, 3])).__name__)
print(type(pd.DataFrame({"a": [1, 2]})).__name__)
```

OUTPUT

```
<exec>:1: DeprecationWarning:
Pyarrow will become a required dependency of pandas in the next major release of pandas
(pandas 3.0),
(to allow more performant data types, such as the Arrow string type, and better
interoperability with other libraries)
but was not found to be installed on your system.
If this would cause problems for you,
please provide us feedback at https://github.com/pandas-dev/pandas/issues/54466

pandas
Series
DataFrame
```

pandas in the Try it boxes

The first box that imports pandas downloads it (a few seconds, more than NumPy). After that it is quick. Some printed details, like the word `object` for text columns, can be different in newer versions of pandas on your own computer.

Series

A **Series** is a one-dimensional column. Each value has a **label** called the **index**.

```
import pandas as pd

marks = pd.Series([85, 90, 78, 92])
print(marks)
```

OUTPUT

```
0    85
1    90
2    78
3    92
dtype: int64
```

- The **left** column (`0 1 2 3`) is the **index**.
- The **right** column is the **values**.
- The last line shows the **dtype** (the data type), here `int64`.

Your own index

```
import pandas as pd

marks = pd.Series([85, 90, 78, 92],
                  index=["Aman", "Ankit", "Neha", "Rohit"],
                  name="maths")
print(marks)
print(list(marks.index))
print(marks.values)
print(marks.name)
```

OUTPUT

```
Aman     85
Ankit    90
Neha     78
Rohit    92
Name: maths, dtype: int64
['Aman', 'Ankit', 'Neha', 'Rohit']
[85 90 78 92]
maths
```

- `index=` gives each value a **name** instead of a number.
- `name=` names the Series (it becomes the column name in a table).
- `.index` and `.values` give the two parts.

From a dictionary

A dictionary becomes a Series: the **keys** are the index and the **values** are the data.

```
import pandas as pd

stock = pd.Series({"apples": 30, "bananas": 12, "cherries": 45})
print(stock)
```

OUTPUT

```
apples      30
bananas     12
cherries    45
dtype: int64
```

A single value for all

```
import pandas as pd

print(pd.Series(0, index=["a", "b", "c"]))
print(pd.Series(range(1, 6)))
```

OUTPUT

```
a    0
b    0
c    0
dtype: int64
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

Using a Series

Getting values

```
import pandas as pd

marks = pd.Series([85, 90, 78, 92],
                  index=["Aman", "Ankit", "Neha", "Rohit"])

print(marks["Neha"])
print(marks.iloc[0])
print(marks[["Aman", "Rohit"]])
print(marks["Ankit":"Rohit"])
```

OUTPUT

```
78
85
Aman     85
Rohit    92
dtype: int64
Ankit    90
Neha     78
Rohit    92
dtype: int64
```

WAY	MEANING
`marks["Neha"]`	by **label**
`marks.loc["Neha"]`	by label (the clear way)
`marks.iloc[2]`	by **position** (0, 1, 2 ...)
`marks[["Aman", "Rohit"]]`	several labels, in a list
`marks["Ankit":"Rohit"]`	a label slice: **includes** the last label

Label slices include the end

A normal slice like `a[1:3]` stops **before** 3. A **label** slice like `"Ankit":"Rohit"` **includes** `"Rohit"`.

Maths and filtering

A Series works like a NumPy array: maths happens on every value, and a condition works as a filter.

```
import pandas as pd

marks = pd.Series([85, 90, 78, 92],
                  index=["Aman", "Ankit", "Neha", "Rohit"])

print(marks + 5)
print(marks[marks > 85])
print(marks.sum(), marks.mean(), marks.max())
print(marks.idxmax())
```

OUTPUT

```
Aman     90
Ankit    95
Neha     83
Rohit    97
dtype: int64
Ankit    90
Rohit    92
dtype: int64
345 86.25 92
Rohit
```

- `marks + 5` keeps the labels and adds 5 to each value.
- `marks[marks > 85]` keeps only the matching rows.
- `idxmax()` gives the **label** of the biggest value (the topper's name).

Series with Series

When you add two Series, pandas lines the values up by **label**, not by position.

```
import pandas as pd

a = pd.Series([1, 2, 3], index=["x", "y", "z"])
b = pd.Series([10, 20, 30], index=["z", "y", "x"])

print(a + b)
```

OUTPUT

```
x    31
y    22
z    13
dtype: int64
```

DataFrame

A **DataFrame** is a table: **rows** and **columns**, each column its own Series. The easiest way to make one is from a **dictionary**: each key is a column name, each value is a list for that column.

```
import pandas as pd

data = {
    "name":    ["Aman", "Ankit", "Neha", "Rohit"],
    "maths":   [85, 70, 92, 55],
    "science": [90, 65, 88, 60],
}

df = pd.DataFrame(data)
print(df)
```

OUTPUT

```
    name  maths  science
0   Aman     85       90
1  Ankit     70       65
2   Neha     92       88
3  Rohit     55       60
```

- The **bold column on the left** (`0 1 2 3`) is the row **index**.
- The top row has the **column names**.
- All the lists must be the **same length**.

The parts of a DataFrame

```
import pandas as pd

df = pd.DataFrame({
    "name":    ["Aman", "Ankit", "Neha", "Rohit"],
    "maths":   [85, 70, 92, 55],
    "science": [90, 65, 88, 60],
})

print(df.shape)
print(list(df.columns))
print(list(df.index))
print(df.size)
```

OUTPUT

```
(4, 3)
['name', 'maths', 'science']
[0, 1, 2, 3]
12
```

ATTRIBUTE	MEANING
`df.shape`	`(rows, columns)`
`df.columns`	the column names
`df.index`	the row labels
`df.dtypes`	the type of each column
`df.size`	rows times columns
`df.values`	the data as a NumPy array

```
import pandas as pd

df = pd.DataFrame({"maths": [85, 70, 92], "ratio": [0.5, 0.25, 0.75]})

print(df.dtypes)
print(df.values)
```

OUTPUT

```
maths      int64
ratio    float64
dtype: object
[[85.    0.5 ]
 [70.    0.25]
 [92.    0.75]]
```

A column of **text** shows the dtype `object`. (Some newer versions of pandas call it `str`.)

Creating a DataFrame in other ways

A list of dictionaries (one per row)

This matches how data often arrives: each record is a dictionary.

```
import pandas as pd

rows = [
    {"name": "Aman",  "city": "Delhi", "age": 21},
    {"name": "Neha",  "city": "Pune",  "age": 23},
    {"name": "Rohit", "city": "Goa"},
]

df = pd.DataFrame(rows)
print(df)
```

OUTPUT

```
    name   city   age
0   Aman  Delhi  21.0
1   Neha   Pune  23.0
2  Rohit    Goa   NaN
```

Rohit has no `age`, so pandas fills the gap with `NaN` (not a number): the sign of a **missing value**.

A list of lists, with column names

```
import pandas as pd

rows = [
    ["Aman", 85, 90],
    ["Ankit", 70, 65],
    ["Neha", 92, 88],
]

df = pd.DataFrame(rows, columns=["name", "maths", "science"])
print(df)
```

OUTPUT

```
    name  maths  science
0   Aman     85       90
1  Ankit     70       65
2   Neha     92       88
```

A NumPy array

```
import numpy as np
import pandas as pd

arr = np.array([[1, 2, 3],
                [4, 5, 6]])

df = pd.DataFrame(arr, columns=["a", "b", "c"], index=["row1", "row2"])
print(df)
```

OUTPUT

```
      a  b  c
row1  1  2  3
row2  4  5  6
```

Your own row labels

```
import pandas as pd

df = pd.DataFrame(
    {"maths": [85, 70, 92], "science": [90, 65, 88]},
    index=["Aman", "Ankit", "Neha"],
)
print(df)
print(df.loc["Neha"])
```

OUTPUT

```
       maths  science
Aman      85       90
Ankit     70       65
Neha      92       88
maths      92
science    88
Name: Neha, dtype: int64
```

Series together make a DataFrame

```
import pandas as pd

maths = pd.Series([85, 70, 92], index=["Aman", "Ankit", "Neha"])
science = pd.Series([90, 65, 88], index=["Aman", "Ankit", "Neha"])

df = pd.DataFrame({"maths": maths, "science": science})
print(df)
```

OUTPUT

```
       maths  science
Aman      85       90
Ankit     70       65
Neha      92       88
```

Creating data with code

You do not have to type every value. Python and NumPy can **generate** the data.

```
import pandas as pd

df = pd.DataFrame({
    "n": range(1, 6),
    "square": [n ** 2 for n in range(1, 6)],
    "double": [n * 2 for n in range(1, 6)],
})
print(df)
```

OUTPUT

```
   n  square  double
0  1       1       2
1  2       4       4
2  3       9       6
3  4      16       8
4  5      25      10
```

Random data

```
import numpy as np
import pandas as pd

rng = np.random.default_rng()
df = pd.DataFrame(rng.integers(35, 101, size=(4, 3)),
                  columns=["maths", "science", "english"])
print(df.shape)
print((df >= 35).all().all())
```

OUTPUT

```
(4, 3)
True
```

A range of dates

```
import pandas as pd

days = pd.date_range("2026-01-01", periods=4)
sales = pd.Series([120, 80, 150, 95], index=days)
print(sales)
```

OUTPUT

```
2026-01-01    120
2026-01-02     80
2026-01-03    150
2026-01-04     95
Freq: D, dtype: int64
```

From CSV text

Real data usually comes from a **CSV file**: `pd.read_csv("marks.csv")`. This box uses text that looks like a file.

```
import pandas as pd
from io import StringIO

text = """name,maths,science
Aman,85,90
Ankit,70,65
Neha,92,88"""

df = pd.read_csv(StringIO(text))
print(df)
print(df["maths"].mean())
```

```
OUTPUT
    name  maths  science
0   Aman     85       90
1  Ankit     70       65
2   Neha     92       88
82.33333333333333
```

Next class we read real CSV files. Today, remember the **names**: `read_csv` to read, and `to_csv` to write.

```
import pandas as pd

df = pd.DataFrame({"name": ["Aman", "Neha"], "marks": [85, 92]})

df.to_csv("marks.csv", index=False)
print(open("marks.csv").read())

back = pd.read_csv("marks.csv")
print(back.equals(df))
```

```
OUTPUT
name,marks
Aman,85
Neha,92

True
```

`index=False` stops the row numbers being saved as an extra column.

Looking at a DataFrame

METHOD	WHAT IT SHOWS
`df.head(n)`	the first `n` rows (5 if you give no number)
`df.tail(n)`	the last `n` rows
`df.describe()`	count, mean, std, min, quartiles and max of the number columns
`df.info()`	columns, types and missing values
`df.sample(n)`	`n` random rows

```
import pandas as pd

df = pd.DataFrame({
    "name":    ["Aman", "Ankit", "Neha", "Rohit", "Sara", "Tara"],
    "maths":   [85, 70, 92, 55, 78, 66],
    "science": [90, 65, 88, 60, 82, 71],
})

print(df.head(3))
print(df.tail(2))
print(df.describe())
```

OUTPUT

```
    name  maths  science
0   Aman     85       90
1  Ankit     70       65
2   Neha     92       88
   name  maths  science
4  Sara     78       82
5  Tara     66       71
           maths    science
count   6.000000   6.000000
mean   74.333333  76.000000
std    13.426342  12.473973
min    55.000000  60.000000
25%    67.000000  66.500000
50%    74.000000  76.500000
75%    83.250000  86.500000
max    92.000000  90.000000
```

- `head` and `tail` are the first thing to run on any new data.
- `describe` skips the text column and summarises the numbers.

Selecting columns and rows

Columns

```
import pandas as pd

df = pd.DataFrame({
    "name":    ["Aman", "Ankit", "Neha"],
    "maths":   [85, 70, 92],
    "science": [90, 65, 88],
})

print(df["maths"])
print(type(df["maths"]).__name__)
print(df[["name", "maths"]])
print(type(df[["name", "maths"]]).__name__)
```

```
OUTPUT
0    85
1    70
2    92
Name: maths, dtype: int64
Series
    name  maths
0   Aman     85
1  Ankit     70
2   Neha     92
DataFrame
```

- `df["maths"]` (one name) gives a **Series**.
- `df[["name", "maths"]]` (a **list** of names, so **double brackets**) gives a **DataFrame**.
- `df.maths` also works, but only when the name has no spaces. Prefer `df["maths"]`.

Rows with iloc and loc

```
import pandas as pd

df = pd.DataFrame({
    "maths":   [85, 70, 92, 55],
    "science": [90, 65, 88, 60],
}, index=["Aman", "Ankit", "Neha", "Rohit"])

print(df.iloc[0])
print(df.loc["Neha"])
print(df.iloc[1:3])
print(df.loc["Aman":"Neha", "maths"])
print(df.iloc[2, 1])
```

OUTPUT

```
maths      85
science    90
Name: Aman, dtype: int64
maths      92
science    88
Name: Neha, dtype: int64
       maths  science
Ankit     70       65
Neha      92       88
Aman     85
Ankit    70
Neha     92
Name: maths, dtype: int64
88
```

TOOL	SELECTS BY	EXAMPLE
`iloc`	**position**	`df.iloc[0]`, `df.iloc[1:3]`, `df.iloc[2, 1]`
`loc`	**label**	`df.loc["Neha"]`, `df.loc["Aman":"Neha", "maths"]`

Both take `[rows, columns]`, just like NumPy. `iloc` slices **stop before** the end; `loc` slices **include** the end.

Filtering rows

```
import pandas as pd

df = pd.DataFrame({
    "name":  ["Aman", "Ankit", "Neha", "Rohit"],
    "maths": [85, 70, 92, 55],
})

print(df[df["maths"] >= 80])
print(df[(df["maths"] > 60) & (df["maths"] < 90)])
```

OUTPUT

```
   name  maths
0  Aman     85
2  Neha     92
    name  maths
0   Aman     85
1  Ankit     70
```

The same rules as NumPy: use `&` and `|`, with each condition in brackets.

Adding, changing and removing columns

```
import pandas as pd

df = pd.DataFrame({
    "name":    ["Aman", "Ankit", "Neha"],
    "maths":   [85, 70, 92],
    "science": [90, 65, 88],
})

df["total"] = df["maths"] + df["science"]
df["average"] = df["total"] / 2
df["passed"] = df["average"] >= 75
print(df)
```

OUTPUT

```
    name  maths  science  total  average  passed
0   Aman     85       90    175     87.5    True
1  Ankit     70       65    135     67.5   False
2   Neha     92       88    180     90.0    True
```

- Assign to a **new** column name to **add** a column.
- The maths is done on the **whole columns** at once, like NumPy.
- Assigning to an **old** name **changes** that column.

Rename and drop

```
import pandas as pd

df = pd.DataFrame({
    "name":  ["Aman", "Ankit", "Neha"],
    "maths": [85, 70, 92],
    "temp":  [0, 0, 0],
})

df = df.rename(columns={"maths": "math_marks"})
df = df.drop(columns=["temp"])
print(df)

df = df.drop(index=[1])
print(df)
```

OUTPUT

```
    name  math_marks
0   Aman          85
1  Ankit          70
2   Neha          92
   name  math_marks
0  Aman          85
2  Neha          92
```

These make a new table

`rename` and `drop` give back a **new** DataFrame. Save it with `df = ...`, or the change is lost.

Sorting

```
import pandas as pd

df = pd.DataFrame({
    "name":  ["Aman", "Ankit", "Neha", "Rohit"],
    "maths": [85, 70, 92, 55],
})

print(df.sort_values("maths"))
print(df.sort_values("maths", ascending=False).head(2))
```

OUTPUT

```
    name  maths
3  Rohit     55
1  Ankit     70
0   Aman     85
2   Neha     92
   name  maths
2  Neha     92
0  Aman     85
```

Working with the index

Any column can become the **index**, and the index can go back to being a column.

```
import pandas as pd

df = pd.DataFrame({
    "name":  ["Aman", "Ankit", "Neha"],
    "maths": [85, 70, 92],
})

by_name = df.set_index("name")
print(by_name)
print(by_name.loc["Neha", "maths"])

print(by_name.reset_index())
```

OUTPUT

```
       maths
name
Aman      85
Ankit     70
Neha      92
92
    name  maths
0   Aman     85
1  Ankit     70
2   Neha     92
```

- `set_index("name")` makes `name` the row labels.
- `reset_index()` puts them back as a normal column and numbers the rows `0, 1, 2 ...`.

Missing values (NaN)

Real data has gaps. pandas shows a gap as `NaN` (or `None`).

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name":  ["Aman", "Ankit", "Neha"],
    "maths": [85, np.nan, 92],
})

print(df)
print(df.isna())
print(df["maths"].isna().sum())
print(df["maths"].mean())
```

OUTPUT

```
    name  maths
0   Aman   85.0
1  Ankit    NaN
2   Neha   92.0
    name  maths
0  False  False
1  False   True
2  False  False
1
88.5
```

- `isna()` gives `True` where a value is missing.
- `isna().sum()` counts the gaps.
- `mean()` **skips** the `NaN` values. The mean here is `(85 + 92) / 2`.

Fill or drop

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "name":  ["Aman", "Ankit", "Neha"],
    "maths": [85, np.nan, 92],
})

print(df.fillna(0))
print(df["maths"].fillna(df["maths"].mean()))
print(df.dropna())
```

OUTPUT

```
    name  maths
0   Aman   85.0
1  Ankit    0.0
2   Neha   92.0
0    85.0
1    88.5
2    92.0
Name: maths, dtype: float64
   name  maths
0  Aman   85.0
2  Neha   92.0
```

METHOD	WHAT IT DOES
`fillna(value)`	replace each gap with `value`
`dropna()`	remove every row that has a gap
`isna()`	find the gaps

Example: a class report

Put it together: build a table, add columns, rank, and find the topper.

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    "name":    ["Aman", "Ankit", "Neha", "Rohit", "Sara"],
    "maths":   [85, 70, 92, 55, 78],
    "science": [90, 65, 88, 60, 82],
    "english": [78, 80, 95, 48, 74],
})

df["total"] = df["maths"] + df["science"] + df["english"]
df["average"] = (df["total"] / 3).round(1)
df["grade"] = np.where(df["average"] >= 80, "A",
              np.where(df["average"] >= 60, "B", "C"))

ranked = df.sort_values("total", ascending=False)
print(ranked[["name", "total", "average", "grade"]])

print("Topper:", ranked.iloc[0]["name"])
print("Class average:", round(df["average"].mean(), 1))
print("Students with grade A:", (df["grade"] == "A").sum())
```

OUTPUT

```
    name  total  average grade
2   Neha    275     91.7     A
0   Aman    253     84.3     A
4   Sara    234     78.0     B
1  Ankit    215     71.7     B
3  Rohit    163     54.3     C
Topper: Neha
Class average: 76.0
Students with grade A: 2
```

- Columns are added with simple maths on other columns.
- `np.where` fills the grade column, as in the NumPy class.
- `sort_values` ranks the table, and `iloc[0]` takes the top row.

Your turn: add a column "percent" (total out of 300, rounded to 1 decimal) and print the table

```
import pandas as pd

df = pd.DataFrame({
    "name":    ["Aman", "Neha"],
    "maths":   [85, 92],
    "science": [90, 88],
    "english": [78, 95],
})
df["total"] = df["maths"] + df["science"] + df["english"]
```

Common mistakes

- **Forgetting** `import pandas as pd`**.** You get `NameError: name 'pd' is not defined`.
- **A wrong column name.** `df["Maths"]` is not `df["maths"]`: you get a `KeyError`.
- **One bracket instead of two** when choosing several columns: `df["name", "maths"]` fails. Use `df[["name", "maths"]]`.
- **Lists of different lengths** when making a DataFrame from a dictionary.
- **Forgetting to save** the result of `drop`, `rename` or `sort_values`: `df = df.drop(...)`.
- **Mixing up** `loc` **and** `iloc`**.** `loc` uses **labels** and includes the end; `iloc` uses **positions** and stops before the end.
- **Using** `and` **/** `or` instead of `&` / `|` when filtering rows.
- **Capital letters.** It is `DataFrame` and `Series`, not `Dataframe` or `series`.

Try it: this fails on purpose

```
import pandas as pd

df = pd.DataFrame({"name": ["Aman"], "maths": [85]})
print(df["Maths"])
```

```
ERROR
  File "your code", line 4, in <module>
  File "/lib/python3.12/site-packages/pandas/core/frame.py", line 4090, in __getitem__
    indexer = self.columns.get_loc(key)
              ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/lib/python3.12/site-packages/pandas/core/indexes/base.py", line 3809, in get_loc
    raise KeyError(key) from err
KeyError: 'Maths'
```

Try it: this also fails on purpose

```
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5]})
print(df)
```

```
ERROR
  File "your code", line 3, in <module>
  File "/lib/python3.12/site-packages/pandas/core/frame.py", line 767, in __init__
    mgr = dict_to_mgr(data, index, columns, dtype=dtype, copy=copy, typ=manager)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/lib/python3.12/site-packages/pandas/core/internals/construction.py", line 503, in
dict_to_mgr
    return arrays_to_mgr(arrays, columns, index, dtype=dtype, typ=typ, consolidate=copy)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/lib/python3.12/site-packages/pandas/core/internals/construction.py", line 114, in
arrays_to_mgr
    index = _extract_index(arrays)
            ^^^^^^^^^^^^^^^^^^^^^^
  File "/lib/python3.12/site-packages/pandas/core/internals/construction.py", line 677, in
_extract_index
    raise ValueError("All arrays must be of the same length")
ValueError: All arrays must be of the same length
```

Practice

Your turn: make a Series of 4 city populations (any numbers) with the city names as the index, then print the biggest city

```
import pandas as pd


```

Your turn: make a DataFrame of 3 books with columns title, author and price, then print only the books costing more than 300

```
import pandas as pd


```

Your turn: add a column "price_with_gst" (price times 1.18) and sort the table by it, biggest first

```
import pandas as pd

df = pd.DataFrame({
    "item":  ["pen", "bag", "book"],
    "price": [20, 800, 350],
})
```

Homework and wrap-up

- Make a DataFrame of **6 students** with name, age, city and marks. Print `shape`, `head(3)` and `describe()`.
- From it, print the students **older than 20**, then only the **name and marks** columns of those students.
- Add a column `result` that says "Pass" or "Fail" using `np.where`, then count how many of each.
- Create the same table from a **list of dictionaries**, then again from a **list of lists** with `columns=`.
- Write your DataFrame to `students.csv` with `to_csv`, then read it back with `read_csv` and check with `equals`.
- Make a table with one **missing value**, find it with `isna`, then fix it with `fillna`.
- Revise: **Series**, **DataFrame**, `index`, `columns`, `loc`, `iloc`, `head`, `describe`, `NaN`.

Next class

Pandas data analysis: reading CSV files, filtering, `groupby` and summaries.